

رسالة محمد



یادگیری ماشین

مدرس: محمدرضا محمدی

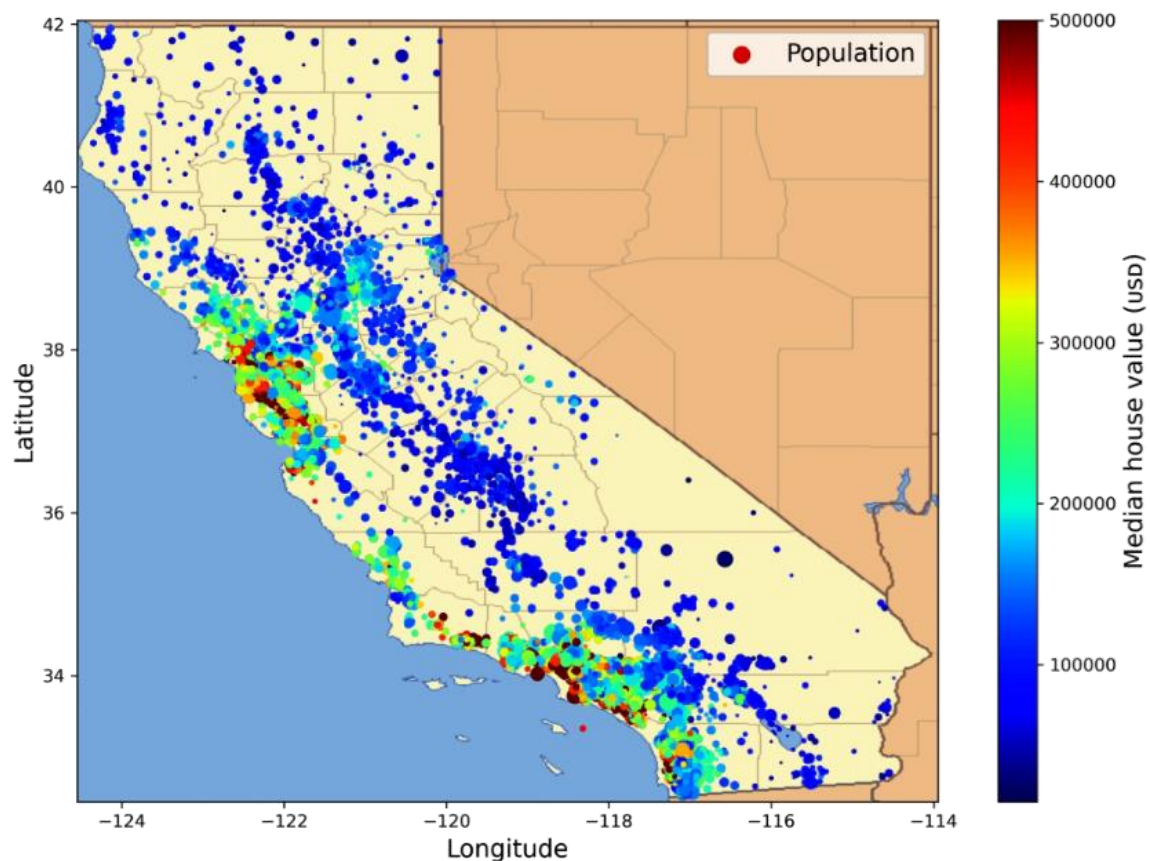
زمستان ۱۴۰۱

پروژه یادگیری ماشین انتها به انتها

End-to-End Machine Learning Project

پروژه یادگیری ماشین انتها به انتها

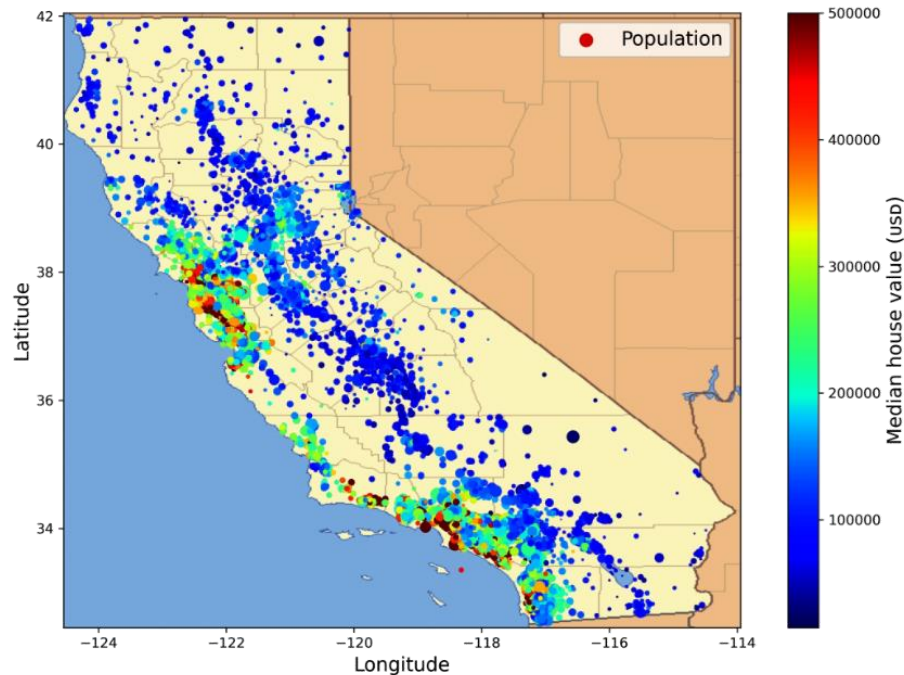
- می‌خواهیم با یک مثال ساختگی (تخمین قیمت مسکن) مراحل اصلی یک پروژه یادگیری ماشین را بررسی کنیم



- نگاه به تصویر بزرگ (big picture)
- دریافت داده‌ها
- بررسی و مصورسازی داده‌ها
- آماده‌سازی داده‌ها برای الگوریتم‌های یادگیری ماشین
- انتخاب مدل و آموزش آن
- تنظیم دقیق مدل
- ارائه راه‌حل پیشنهادی
- راه‌اندازی، نظارت و نگهداری سیستم

نگاه به تصویر بزرگ

- مجموعه داده از سرشماری سال ۱۹۹۰ کالیفرنیا تهیه شده است
- داده‌ها شامل معیارهایی مانند جمعیت، درآمد متوسط و قیمت متوسط مسکن برای هر محله است
 - محله کوچکترین واحد جغرافیایی است که داده‌های آن منتشر شده است
 - یک محله به طور معمول بین ۶۰۰ تا ۳۰۰۰ نفر جمعیت دارد
- مدل باید از این داده‌ها بیاموزد و بتواند میانگین قیمت مسکن در هر محله را با توجه به تمام معیارهای دیگر پیش‌بینی کند
 - یادگیری با نظارت
 - رگرسیون یک‌متغیره



معیار عملکرد (performance measure)

- یک معیار متداول برای مسائل رگرسیون، ریشه میانگین مربعات خطا (RMSE) است

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m (h(\mathbf{x}^{(i)}) - y^{(i)})^2}$$

- نمادهای مورد استفاده:

- m تعداد نمونه‌ها در مجموعه داده

- $\mathbf{x}^{(i)}$ تمام ویژگی‌های نمونه i ام (به غیر از برچسب)

▪ به عنوان نمونه، محلّهای در موقعیت جغرافیایی -118.29° و 33.91° با تعداد 1,416 ساکن و درآمد متوسط $38,372\$$

- $y^{(i)}$ برچسب نمونه i ام

▪ به عنوان نمونه، محلّهای با قیمت مسکن متوسط $156,400\$$

$$\mathbf{x}^{(1)} = \begin{pmatrix} -118.29 \\ 33.91 \\ 1,416 \\ 38,372 \end{pmatrix}$$

$$y^{(1)} = 156,400$$

معیار عملکرد (performance measure)

- یک معیار متداول برای مسائل رگرسیون، ریشه میانگین مربعات خطا (RMSE) است

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m (h(\mathbf{x}^{(i)}) - y^{(i)})^2}$$

- نمادهای مورد استفاده:

- m تعداد نمونه‌ها در مجموعه داده

- $\mathbf{x}^{(i)}$ تمام ویژگی‌های نمونه i ام (به غیر از برچسب)

- $y^{(i)}$ برچسب نمونه i ام

- $\mathbf{X}$ تمام ویژگی‌های تمام نمونه‌ها

- h تابع پیش‌بینی سیستم

$$\mathbf{x}^{(1)} = \begin{pmatrix} -118.29 \\ 33.91 \\ 1,416 \\ 38,372 \end{pmatrix} \quad \mathbf{X} = \begin{pmatrix} (\mathbf{x}^{(1)})^\top \\ (\mathbf{x}^{(2)})^\top \\ \vdots \\ (\mathbf{x}^{(1999)})^\top \\ (\mathbf{x}^{(2000)})^\top \end{pmatrix} = \begin{pmatrix} -118.29 & 33.91 & 1,416 & 38,372 \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix}$$
$$y^{(1)} = 156,400$$

معیار عملکرد (performance measure)

- یک معیار متداول برای مسائل رگرسیون، ریشه میانگین مربعات خطا (RMSE) است

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m (h(\mathbf{x}^{(i)}) - y^{(i)})^2}$$

- اگرچه RMSE معمولاً معیار عملکرد ترجیحی برای وظایف رگرسیونی است، در برخی مسائل ممکن است تابع دیگری ترجیح داشته باشد

- به عنوان مثال، اگر داده‌های پرت وجود داشته باشد

- می‌توان از میانگین قدرمطلق خطا (MAE) استفاده کرد

$$\text{MAE}(\mathbf{X}, h) = \frac{1}{m} \sum_{i=1}^m |h(\mathbf{x}^{(i)}) - y^{(i)}|$$

مجموعه داده

- در این مثال، داده‌ها در یک فایل csv ذخیره شده‌اند که به راحتی قابل دانلود است

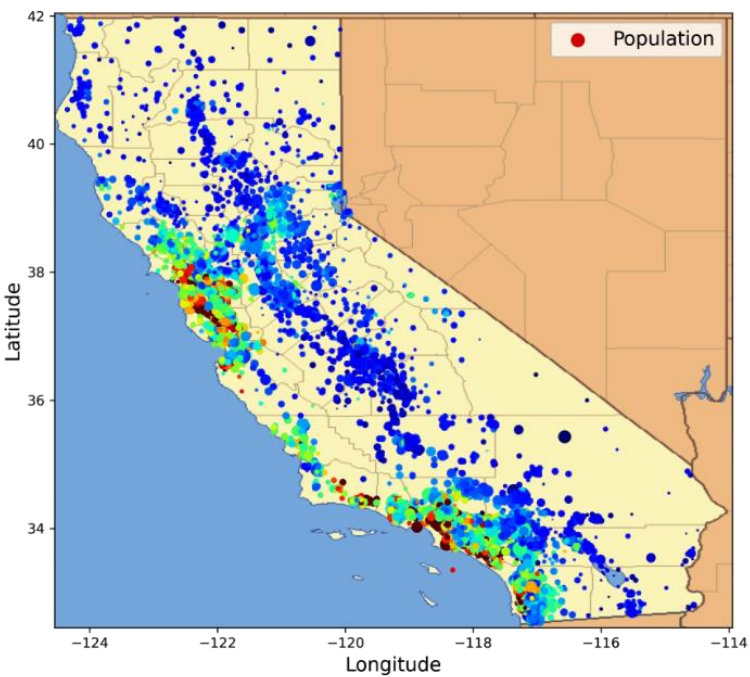
```
from pathlib import Path
import pandas as pd
import tarfile
import urllib.request

def load_housing_data():
    tarball_path = Path("datasets/housing.tgz")
    if not tarball_path.is_file():
        Path("datasets").mkdir(parents=True, exist_ok=True)
        url = "https://github.com/ageron/data/raw/main/housing.tgz"
        urllib.request.urlretrieve(url, tarball_path)
        with tarfile.open(tarball_path) as housing_tarball:
            housing_tarball.extractall(path="datasets")
    return pd.read_csv(Path("datasets/housing/housing.csv"))

housing = load_housing_data()
```

housing.head()

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY



```
>>> housing.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   longitude              20640 non-null  float64
1   latitude               20640 non-null  float64
2   housing_median_age     20640 non-null  float64
3   total_rooms            20640 non-null  float64
4   total_bedrooms        20433 non-null  float64
5   population             20640 non-null  float64
6   households             20640 non-null  float64
7   median_income          20640 non-null  float64
8   median_house_value     20640 non-null  float64
9   ocean_proximity        20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB

>>> housing["ocean_proximity"].value_counts()
<1H OCEAN      9136
INLAND         6551
NEAR OCEAN     2658
NEAR BAY       2290
ISLAND          5
Name: ocean_proximity, dtype: int64
```

مشخصه‌های آماری ویژگی‌ها

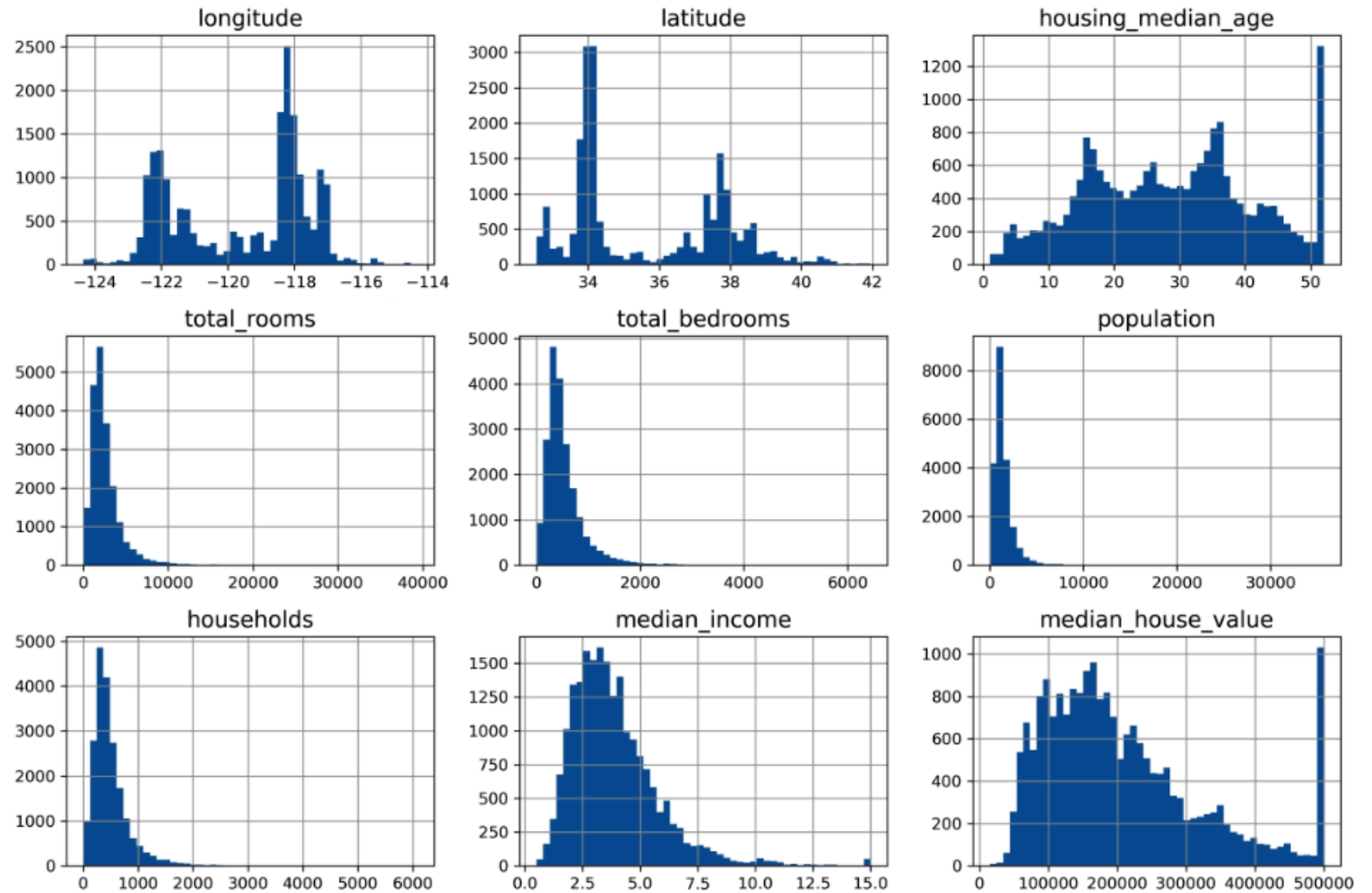
```
housing.describe()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000	20640.000000	20640.000000	20640.000000	20640.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553	1425.476744	499.539680	3.870671	206855.816909
std	2.003532	2.135952	12.585558	2181.615252	421.385070	1132.462122	382.329753	1.899822	115395.615874
min	-124.350000	32.540000	1.000000	2.000000	1.000000	3.000000	1.000000	0.499900	14999.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000	787.000000	280.000000	2.563400	119600.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	1166.000000	409.000000	3.534800	179700.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000	1725.000000	605.000000	4.743250	264725.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	35682.000000	6082.000000	15.000100	500001.000000

هیستوگرام

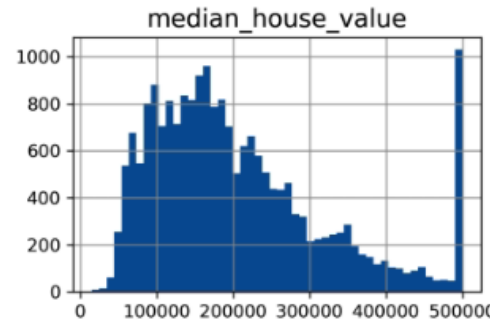
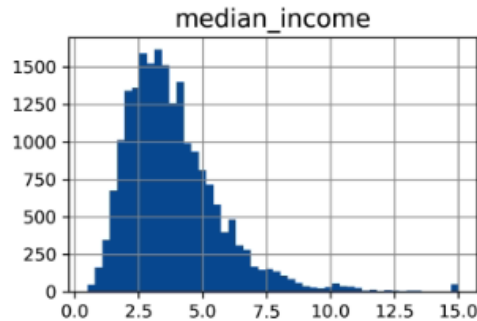
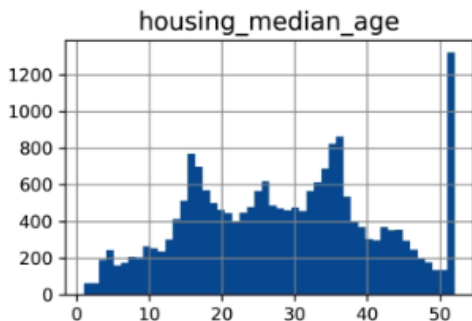
```
import matplotlib.pyplot as plt
```

```
housing.hist(bins=50, figsize=(12, 8))  
plt.show()
```



هیستوگرام

- در جمع‌آوری مجموعه داده، برای برخی مقادیر محدوده قرار داده شده است
 - مثلاً قیمت‌های بیش از \$500,000 برش خورده‌اند
 - به خصوص این موضوع در رابطه هدفی که قرار است پیش‌بینی شود چالش ایجاد می‌کند
 - اگر پیش‌بینی نکردن قیمت‌های بیش از \$500,000 برای کارفرما نامطلوب است:
 - می‌توان قیمت درست را برای چنین محله‌هایی جمع‌آوری کرد
 - می‌توان این داده‌ها را حذف کرد یا تابع ضرر آنها را اصلاح کرد



ایجاد مجموعه ارزیابی

```
import numpy as np
```

```
def shuffle_and_split_data(data, test_ratio):  
    shuffled_indices = np.random.permutation(len(data))  
    test_set_size = int(len(data) * test_ratio)  
    test_indices = shuffled_indices[:test_set_size]  
    train_indices = shuffled_indices[test_set_size:]  
    return data.iloc[train_indices], data.iloc[test_indices]
```

```
>>> train_set, test_set = shuffle_and_split_data(housing, 0.2)
```

```
>>> len(train_set)
```

```
16512
```

```
>>> len(test_set)
```

```
4128
```

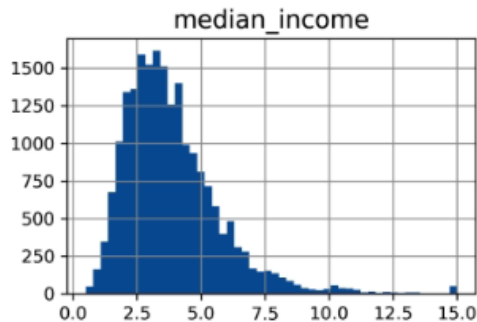
```
from sklearn.model_selection import train_test_split
```

```
train_set, test_set = train_test_split(housing, test_size=0.2, random_state=42)
```

- می‌توان به صورت تصادفی بخشی از داده‌های جمع‌آوری شده را برای ارزیابی جدا کرد
- لازم است داده‌های ارزیابی در اجراهای مختلف یکسان باشند

ایجاد مجموعه ارزیابی

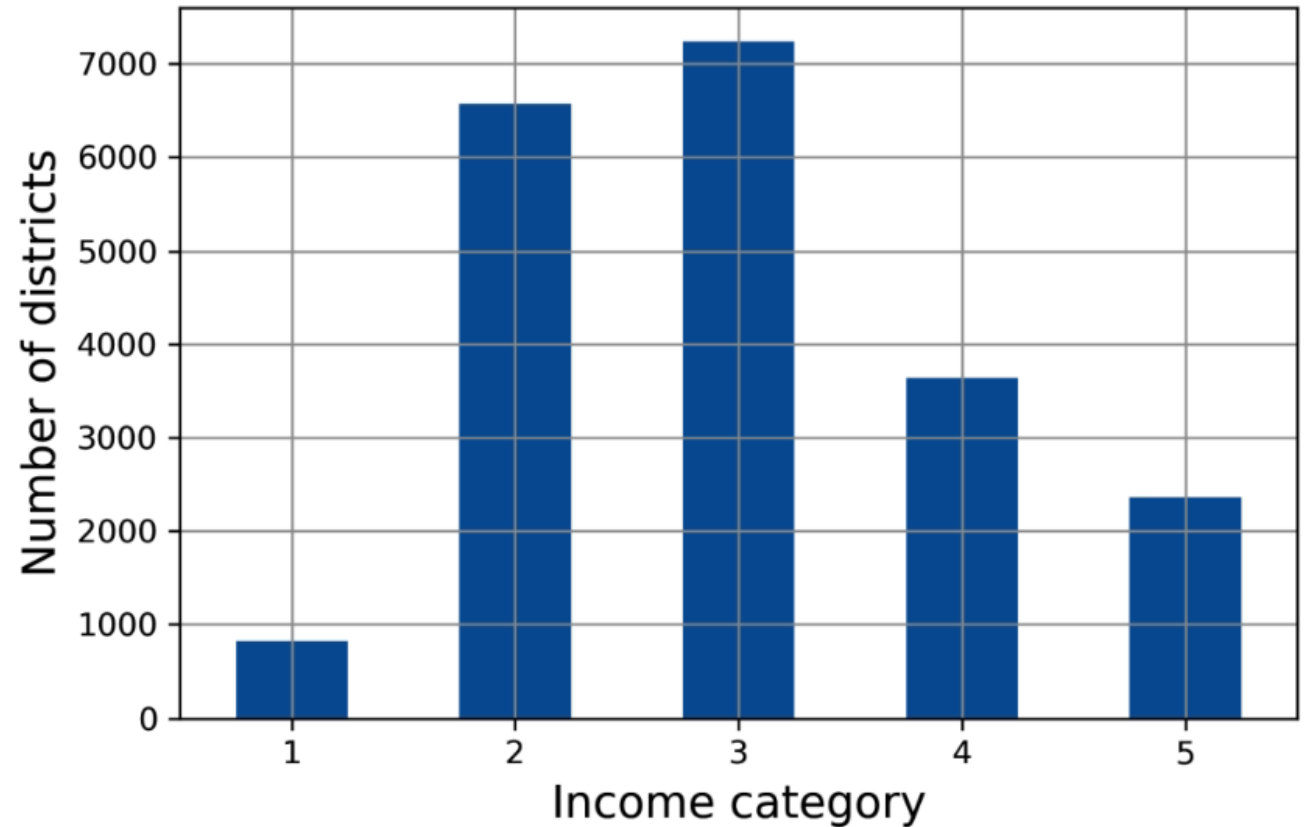
- اگر مجموعه داده به اندازه کافی بزرگ باشد، انتخاب تصادفی مجموعه ارزیابی مناسب است
 - در غیر این صورت، خطر سوگیری نمونه‌برداری وجود دارد
- مجموعه ارزیابی باید به خوبی نماینده داده‌ها باشد
- فرض کنید درآمد متوسط یک ویژگی بسیار مهم برای پیش‌بینی قیمت مسکن متوسط باشد
- از آنجائیکه درآمد متوسط یک ویژگی پیوسته است، می‌توان آن را به چند بخش تقسیم کرد و از هر بخش به تعداد مناسب نمونه‌برداری کرد



```
housing["income_cat"] = pd.cut(housing["median_income"],  
                                bins=[0., 1.5, 3.0, 4.5, 6., np.inf],  
                                labels=[1, 2, 3, 4, 5])
```

ایجاد مجموعه ارزیابی

```
housing["income_cat"].value_counts().sort_index().plot.bar(rot=0, grid=True)
plt.xlabel("Income category")
plt.ylabel("Number of districts")
plt.show()
```



ایجاد مجموعه ارزیابی

```
from sklearn.model_selection import StratifiedShuffleSplit

splitter = StratifiedShuffleSplit(n_splits=10, test_size=0.2, random_state=42)
strat_splits = []
for train_index, test_index in splitter.split(housing, housing["income_cat"]):
    strat_train_set_n = housing.iloc[train_index]
    strat_test_set_n = housing.iloc[test_index]
    strat_splits.append([strat_train_set_n, strat_test_set_n])

strat_train_set, strat_test_set = strat_splits[0]

strat_train_set, strat_test_set = train_test_split(
    housing, test_size=0.2, stratify=housing["income_cat"], random_state=42)
```

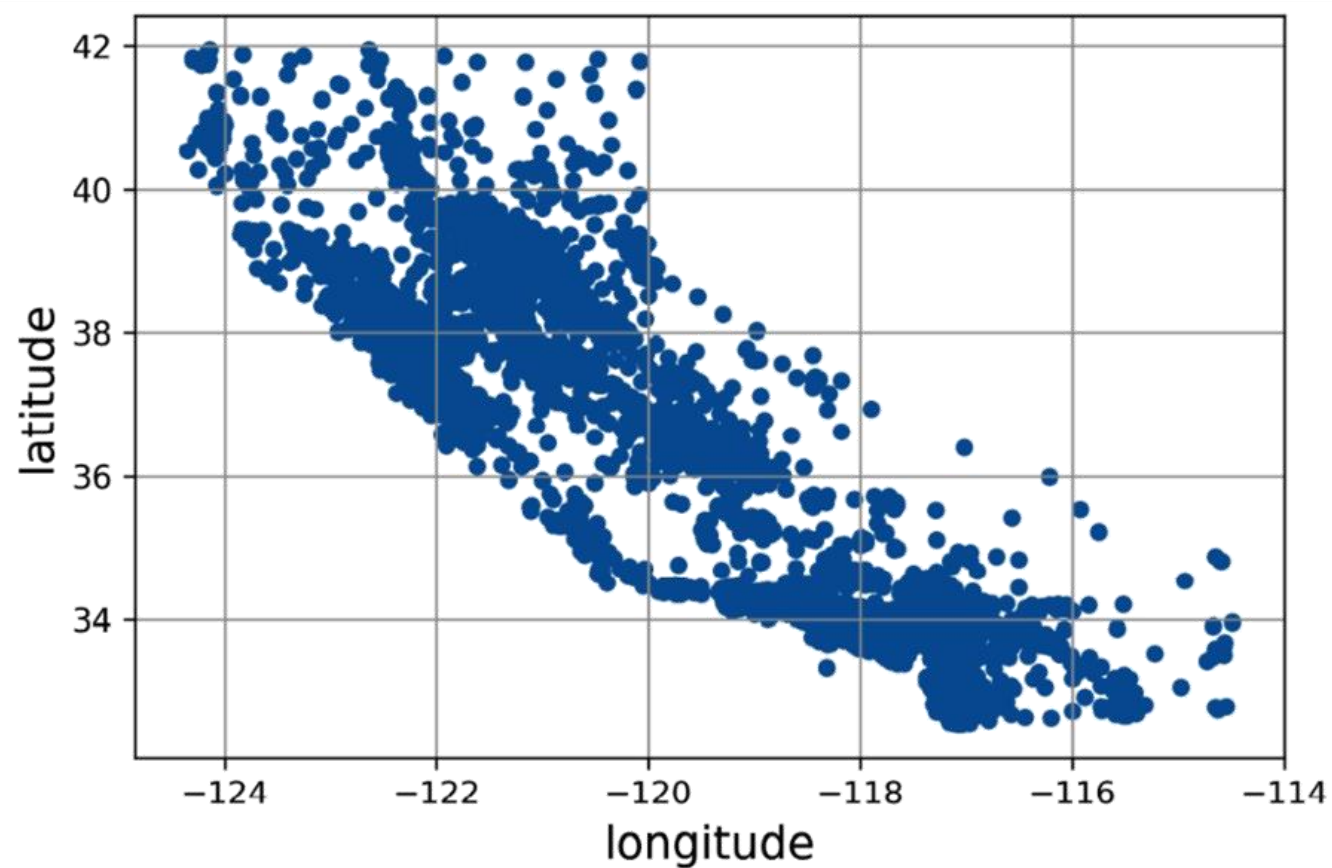
ایجاد مجموعه ارزیابی

```
>>> strat_test_set["income_cat"].value_counts() / len(strat_test_set)
3    0.350533
2    0.318798
4    0.176357
5    0.114341
1    0.039971
Name: income_cat, dtype: float64
```

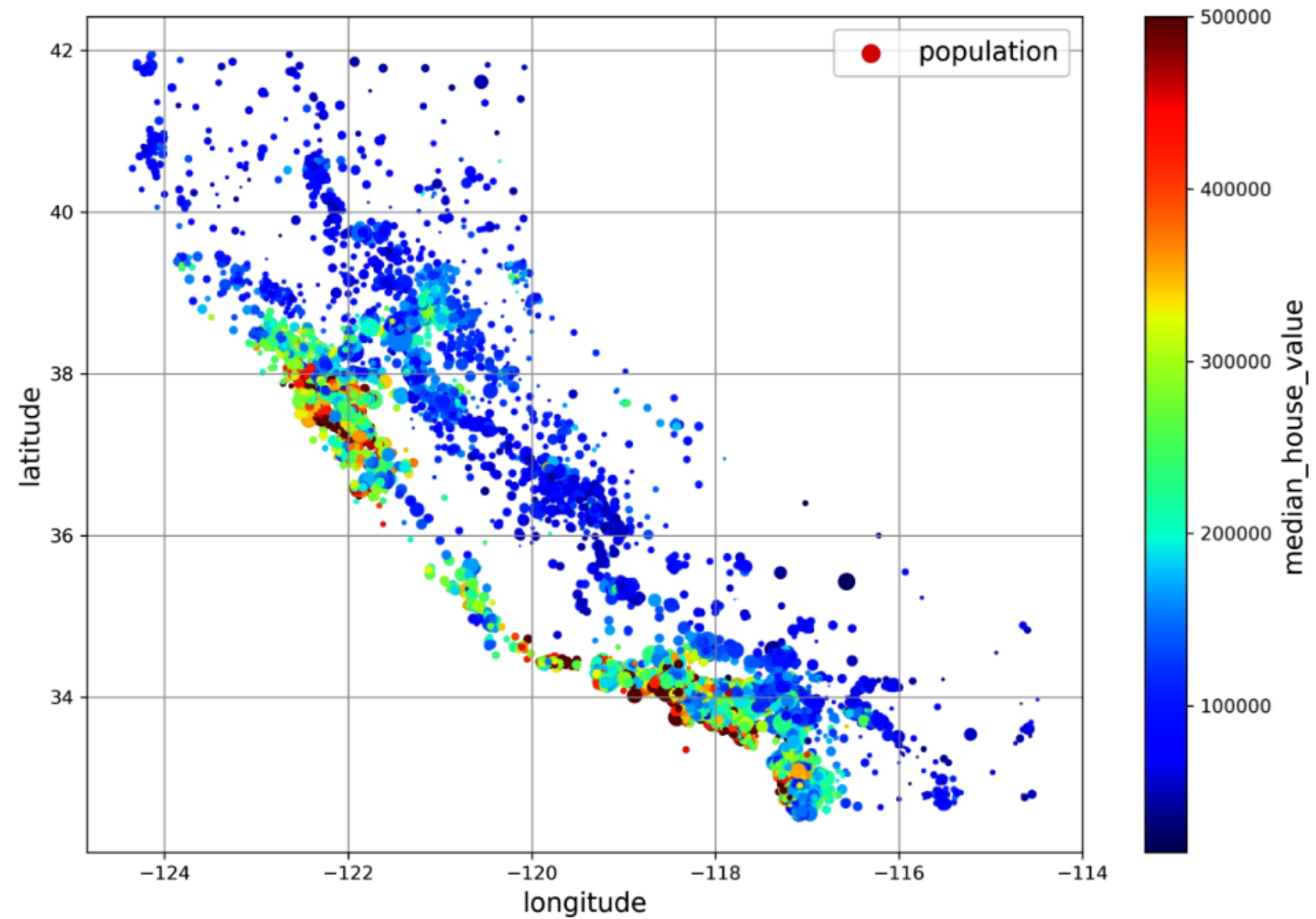
	Overall %	Stratified %	Random %	Strat. Error %	Rand. Error %
Income Category					
1	3.98	4.00	4.24	0.36	6.45
2	31.88	31.88	30.74	-0.02	-3.59
3	35.06	35.05	34.52	-0.01	-1.53
4	17.63	17.64	18.41	0.03	4.42
5	11.44	11.43	12.09	-0.08	5.63

بررسی و مصورسازی داده‌ها

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True)  
plt.show()
```



```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True,  
             s=housing["population"] / 100, label="population",  
             c="median_house_value", cmap="jet", colorbar=True,  
             legend=True, sharex=False, figsize=(10, 7))  
  
plt.show()
```



همبستگی (Correlation)

- ضریب همبستگی از -۱ تا ۱ متغیر است

$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

```
corr_matrix = housing.corr()
```

```
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
```

```
median_house_value    1.000000
median_income          0.688380
total_rooms            0.137455
housing_median_age     0.102175
households             0.071426
total_bedrooms         0.054635
population             -0.020153
longitude              -0.050859
latitude               -0.139584
Name: median_house_value, dtype: float64
```

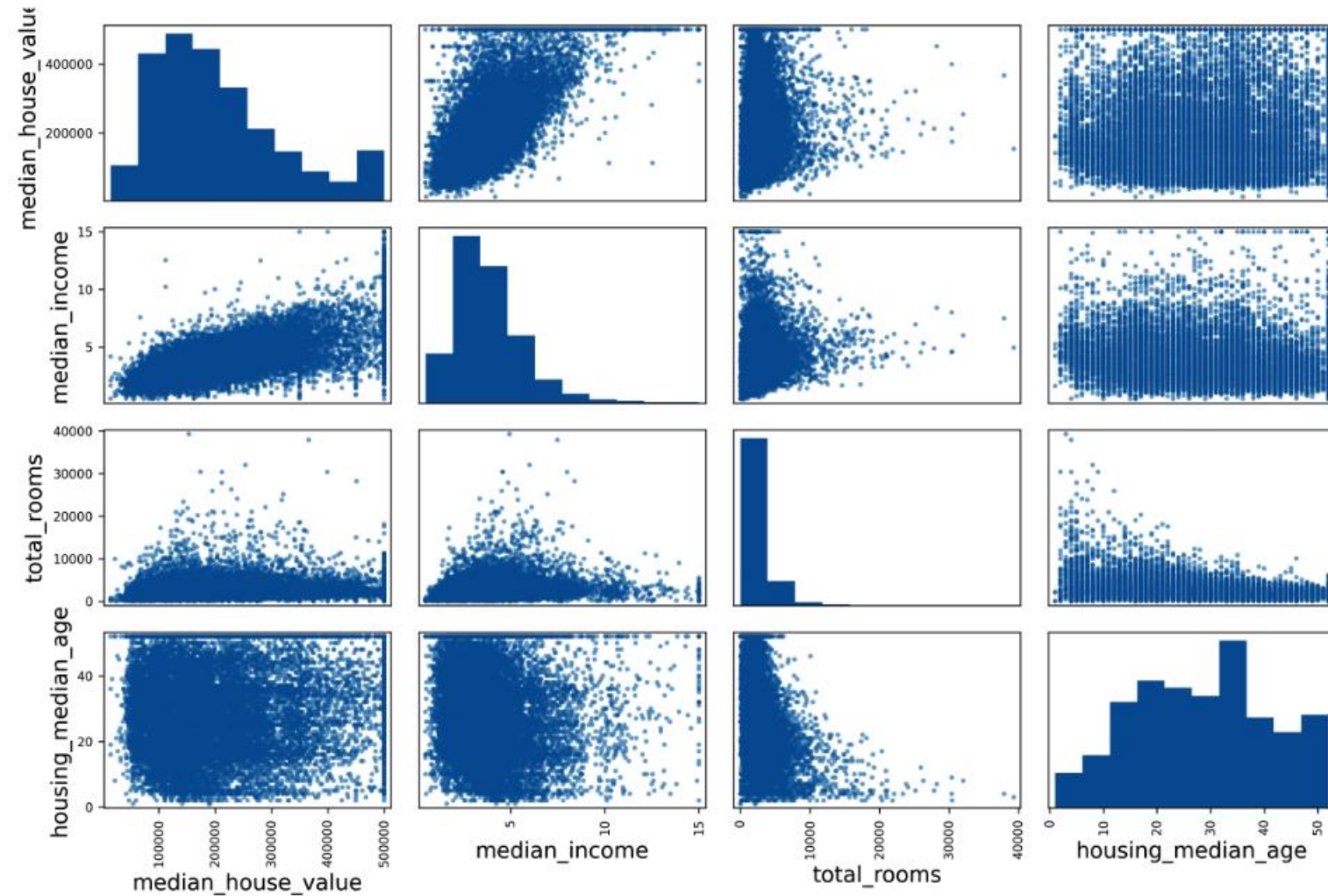


```
from pandas.plotting import scatter_matrix
```

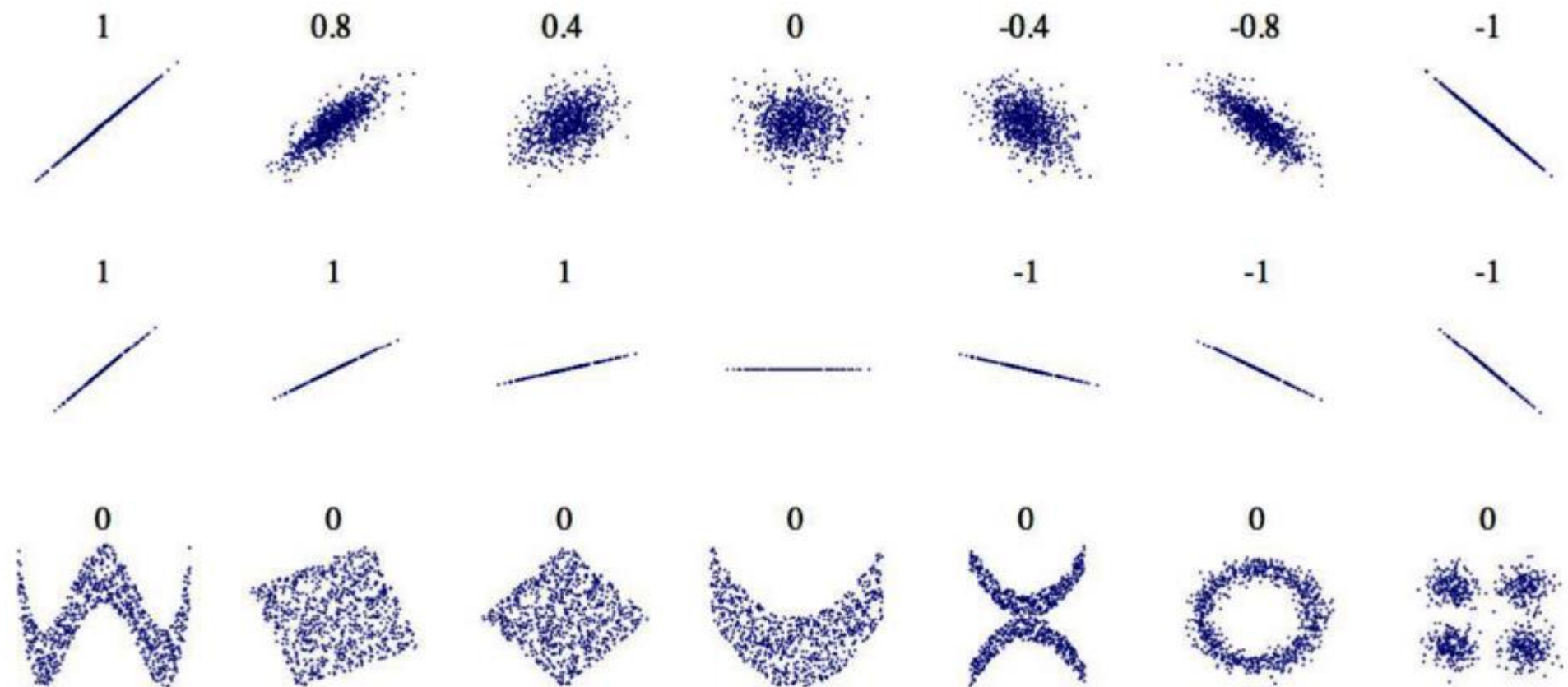
```
attributes = ["median_house_value", "median_income", "total_rooms",  
             "housing_median_age"]
```

```
scatter_matrix(housing[attributes], figsize=(12, 8))
```

```
plt.show()
```



همبستگی



ترکیب ویژگی‌ها

- با ترکیب ویژگی‌ها، می‌توان ویژگی‌های جدید ایجاد کرد که برای حل مسئله مناسب‌تر باشند

```
housing["rooms_per_house"] = housing["total_rooms"] / housing["households"]
housing["bedrooms_ratio"] = housing["total_bedrooms"] / housing["total_rooms"]
housing["people_per_house"] = housing["population"] / housing["households"]
```

```
>>> corr_matrix = housing.corr()
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
```

median_house_value	1.000000
median_income	0.688380
rooms_per_house	0.143663
total_rooms	0.137455
housing_median_age	0.102175
households	0.071426
total_bedrooms	0.054635
population	-0.020153
people_per_house	-0.038224
longitude	-0.050859
latitude	-0.139584
bedrooms_ratio	-0.256397

```
Name: median_house_value, dtype: float64
```